Museum Guard Shift – Draft Report

Group Members: Dmitri Lvovich, Raphaël Michnik

Report

Intended solution and explanation for why it works

The intended solution is:

1. preprocess the tree so that distances between important rooms can be queried quickly,
2. run dynamic programming over subsets of alarmed rooms.

The important rooms are:

- the starting room S,
- the K alarmed rooms.

Because the graph is a tree, the shortest path distance between any two rooms is just the path length in the tree. We can root the tree anywhere, preprocess depths and root-distances, and use lowest common ancestor (LCA) to compute

$$\text{dist}(u, v) = \text{distRoot}[u] + \text{distRoot}[v] - 2 \cdot \text{distRoot}[\text{lca}(u, v)].$$

This lets us build a complete weighted graph on the K alarmed rooms plus the start room, where the edge weights are travel times in the original tree.

Then we define a dynamic programming state

$$dp[\text{mask}][i]$$

where:

- `mask` indicates which alarmed rooms have already been handled,
- i is the last handled alarmed room,
- $dp[\text{mask}][i]$ is the earliest possible time at which the guard can finish handling all alarms in `mask` and end at room i.

Initialization. For each alarmed room i, the guard can go there directly from S. So the initial completion time is

$$\text{dist}(S, i) + a_i.$$

If that time is at most d_i, then

$$dp[1 \ll i][i] = \text{dist}(S, i) + a_i.$$

Otherwise that state is invalid.

Transition. Suppose we are at state $dp[\text{mask}][i]$, and want to handle a new alarm j not in `mask`. The completion time would be

$$dp[\text{mask}][i] + \text{dist}(i, j) + a_j.$$

This transition is valid only if the above quantity is at most d_j. If valid, update

$$dp[\text{mask} \cup \{j\}][j] = \min\Big(dp[\text{mask} \cup \{j\}][j],\ dp[\text{mask}][i] + \text{dist}(i, j) + a_j\Big).$$

Answer. If $K = 0$, the answer is 0. Otherwise, the answer is

$$\min_i dp[(1 \ll K) - 1][i].$$

If all such states are unreachable, print `IMPOSSIBLE`.

Why it works

This dynamic programming is correct because every feasible solution has some last handled alarm. If we know the set of already handled alarms and the last one visited, then the only information needed for future decisions is the current completion time and current room. Since $dp[\text{mask}][i]$ stores the earliest such completion time, it dominates any slower way to reach the same state.

This gives an optimal substructure: every optimal full schedule consists of an optimal schedule for some smaller subset, followed by one final valid transition.

Thus the recurrence explores exactly all feasible orders of alarm handling, while pruning dominated states by keeping only the earliest completion time for each (mask, i).

Complexity

- Tree preprocessing with DFS and LCA preprocessing: $O(N \log N)$
- Pairwise distances between the $K + 1$ important rooms:
  - $O(K^2 \log N)$ with binary lifting
  - $O(K^2)$ with Euler tour + RMQ and $O(1)$ LCA queries
- Dynamic programming: $O(2^K K^2)$, since there are $O(2^K K)$ dp states, one for each pair $(\texttt{mask}, i)$, and each state tries up to K possible next alarms.

This is feasible for $N \leq 200\,000$ and $K \leq 18$.

Any intended problem pitfalls or incorrect approaches

A few natural wrong ideas:

1. **Greedy by nearest room.** Choosing the closest unhandled alarm is not always optimal. A slightly farther room may have a much tighter deadline, so going to the nearest first can make the instance fail even when a valid schedule exists.
2. **Greedy by earliest deadline.** Likewise, always going to the smallest deadline first can fail because the travel time between rooms matters. The order must balance both current position and remaining deadlines.

3. **Inefficient Distance Computation** An inefficient approach is to recompute path lengths from scratch. Because the graph is a tree, we can preprocess distances once and only query the start room and alarmed rooms needed by the dynamic programming.
4. **Treating it as ordinary shortest path.** This is not a plain shortest path problem because the guard must choose an order over multiple required targets, and deadlines constrain which permutations are feasible.
5. **Checking deadline at arrival instead of completion.** The statement says the alarm is handled only after spending a_i time opening the panel, so the deadline applies to finishing the action, not just reaching the room. This can easily be mishandled during implementation.

Any particular design choices

1. **Tree structure with small K.** Using a tree allows large input sizes while keeping distance queries structured, and the small number of alarmed rooms makes the subset dynamic programming feasible.
2. **Weighted edges.** Weighted corridors make the problem less like a simple BFS ordering problem and force contestants to reason about true travel time.
3. **Start room may be alarmed.** This could be a good edge case depending on implementation.
4. **Finish anywhere.** The guard does not need to return to the start. This makes the optimization cleaner and avoids adding a redundant final leg.
5. **Readability.** The statement is intentionally compact: tree, start room, alarm rooms with opening times and deadlines, and minimum finish time or impossible. The goal was to keep the setup easy to follow.

Descriptions of planned test cases needed to verify correctness and efficiency

We would include both hand-crafted and generated tests.

Hand-crafted correctness tests.

- No alarms: $K = 0$, answer should be 0.
- Single alarm, feasible: direct travel plus handling fits deadline.
- Single alarm, impossible: deadline missed even if visited first.
- Start room is alarmed: handling can begin immediately with travel time 0.
- Alarmed room is entered but skipped: alarmed room may have to be entered but skipped to fit within deadline.
- Exact deadline equality: finish time exactly equals deadline, verifying $\leq$ instead of $<$.
- Chain-shaped tree: tests long similar routes.
- Star-shaped tree: all alarms branch from a center, good for exposing greedy mistakes.
- Balanced binary tree: structured but nontrivial distances.

- Must revisit shared ancestors: alarms in different branches, ensuring solutions correctly use tree distances through LCA structure.
- Impossible only because of ordering: every alarm individually reachable, but no global order works.
- Multiple valid orders but different total times: verifies optimization, not just feasibility.
- Greedy trap: nearest-first fails, optimal order succeeds.

Performance and stress tests.

- Large chain-shaped tree: tests tree distance calculation performance.
- Large star-shaped tree: also tests tree distance calculation performance.
- Large balanced binary tree: tests tree distance calculation performance once again.
- Maximum K and N: large distances arranged so many dynamic programming states are reachable.
- Large path lengths: weights large enough to require 64-bit arithmetic.

Plans on generating said test cases

We would generate tests in several ways:

1. **Manual examples.** Small examples written by hand for sample cases, deadline edge cases, greedy-failure constructions, the start-room-alarmed case, etc.
2. **Structured generators.** We would write generators for chains, stars, balanced binary trees, and random trees.
3. **Deadline generation.** For each generated instance, choose an intended visiting order and compute its exact finish times, then create several variants: deadlines comfortably above finish times, deadlines exactly equal to finish times, deadlines lower by 1 on one room to create near-failure, and mixed loose and tight deadlines.

An argument for why those test cases are sufficient

These tests cover both correctness and efficiency.

For correctness, they cover:

- trivial instances,
- edge cases,
- exact-boundary behavior,
- multiple tree shapes,
- cases requiring backtracking across the tree,
- cases where greedy reasoning fails,
- both feasible and impossible instances.

For efficiency, they cover:

- very large trees,
- maximum number of alarms,
- large edge weights,
- random structures.

Taken together, these tests exercise every key aspect of the problem:

- tree distance handling,
- deadline interpretation,
- subset dynamic programming state transitions,
- 64-bit arithmetic,
- asymptotic performance.